

我觉得 **cost-aware** 这条线值得认真做，而且它比“单纯 SQL+CoT 混合”更容易形成清晰贡献。你们现在的项目本来就强调“把大模型能力蒸馏到 2B 以下小模型，提升本地化部署可能性”，但目前主要指标还是 accuracy；如果引入 cost-aware，叙事会更完整：**不是只追求更高正确率，而是让小模型学会在不同成本预算下选择足够好的推理路径。** 这也能避开一部分 Table-R1 / Table-r1 的正面竞争。Table-R1 主打 inference-time scaling、DeepSeek-R1 traces 蒸馏和 RLVR/GRPO；Table-r1 则已经涉及 program-based reasoning 与 text-based reasoning fallback。你们可以把问题改成：**表格推理里的计算路径选择，能否被成本约束显式监督和优化？**

1. 核心研究问题

可以把 topic 定义成：

Cost-aware Path Selection for Small Table Reasoning Models

或者更完整一点：

Cost-aware Paradigm Distillation for Efficient Table Reasoning

核心问题不是“SQL 好还是 CoT 好”，而是：

给定一个表格问题，小模型应该选择哪条推理路径，才能在满足正确率的前提下最小化 token、工具调用、延迟和系统复杂度？

这和你们现有工作是兼容的。你们现在已经有 CoT-only、SQL-only、SQL+CoT mixed 三套轨迹，以及 Qwen3-1.7B 的多个训练版本。报告中也强调当前方法的优势是将大模型能力压缩进 2B 以下模型，并希望提高部署性。cost-aware 正好可以把“部署性”从口号变成可量化指标。

2. 为什么 cost-aware 是更好的切入点

现在很多表格推理工作默认追求最高分，但真实部署里并不是每个问题都值得走最复杂的路径。

比如：

问题类型	最便宜可用路径	没必要使用的路径
单元格查找	Direct Answer	多轮 SQL Agent
简单比较	Direct / Short CoT	Deep CoT + SQL repair
数值聚合	SQL	长 CoT
自由文本描述	CoT	SQL-heavy pipeline
SQL 报错但语义清楚	CoT fallback	多轮 repair
大表筛选聚合	SQL / Program	全表 CoT

你们现在的 mixed 方法是“平均效果好”，但 cost-aware 的目标是：

每个样本只花必要的推理成本。

这个点很适合小模型表格推理，因为 1.7B 模型的核心优势不是绝对能力，而是**便宜、快、可本地化**。如果论文只说 accuracy，可能会被 7B Table-R1 压住；如果论文说 **accuracy-cost Pareto frontier**，你们的 1.7B 反而有空间。

3. 可以定义哪些 cost

不要只把 cost 理解成 token 数。表格推理系统里的 cost 可以分成四类。

3.1 Token cost

包括：

- 输入表格 token 数；
- reasoning token 数；
- answer token 数；
- 多轮 agent 的历史 token 数。

CoT 的主要成本在这里。尤其长 reasoning 对小模型来说不仅慢，还可能增加幻觉。

3.2 Tool cost

包括：

- SQL execution 次数；
- Python execution 次数；

- schema linking 调用;
- retrieval / table trimming 调用;
- repair loop 次数。

SQL 本身生成可能不长，但一旦进入 agent repair loop，工具调用成本会升高。

3.3 Latency cost

包括:

- 模型生成时间;
- SQL 执行时间;
- 表格预处理时间;
- 多轮交互时间。

真实部署时 latency 很关键。一个 SQL Agent 多轮纠错即使 token 不多，也可能比 Direct Answer 慢很多。

3.4 System complexity cost

这是论文里可以提，但实验中不一定容易量化。比如:

- 是否需要外部 SQLite;
- 是否需要 Python sandbox;
- 是否需要额外 parser;
- 是否需要多个 prompt;
- 是否需要 fallback controller。

你们报告里强调方法简单和可部署，但如果系统仍依赖外部执行器和两套 prompt，这个 claim 容易被质疑。所以最好把它显式量化，而不是只说“简单”。

4. 推理路径设计

我建议不要只保留 SQL 和 CoT 两条路径，而是设计一个**成本阶梯**。

可以定义 5 条路径:

路径	成本	适用场景
Direct	最低	单元格读取、简单事实判断
Short-CoT	低	简单比较、轻量语义理解
SQL	中	筛选、排序、聚合、计数
SQL-Repair	高	SQL 初次失败但可修复
CoT-Fallback / Hybrid	高	SQL 不适合或执行失败后转文本推理

如果想更强，可以加 Python：

路径	适用场景
Python Program	复杂计算、非 SQL 友好操作、表格 reshape

但我建议第一版不要太复杂。先做 Direct / CoT / SQL / SQL-Repair / Fallback 五类就够了。

关键是让模型学会：

简单问题不 overthink，数值问题不 hallucinate，SQL 失败时不死磕。

5. 方法框架

可以把方法设计成三阶段。

阶段一：多路径轨迹生成

对每个训练样本，同时生成多条候选路径：

Path 1: Direct Answer
 Path 2: CoT
 Path 3: SQL
 Path 4: SQL + Repair
 Path 5: SQL + CoT Fallback

每条路径都记录：

```
{
  "path_type": "SQL",
  "final_answer": "...",
  "correct": true,
  "token_cost": 183,
```

```
"tool_calls": 1,
"latency": 0.74,
"execution_status": "success",
"error_type": null
}
```

你们现有的 `turn_details` 字段可以直接升级成这种结构化 `trace`。报告里说目前三套蒸馏轨迹数据已经包含完整 `turn_details`，这正好是优势。

阶段二：构造 cost-aware oracle

对每个样本，不再简单选“最高正确率路径”，而是选**满足正确性的最低成本路径**。

可以定义一个 `utility`：

$$\text{Utility}(\text{path}) = \text{Accuracy}(\text{path}) - \lambda_{\text{token}} * \text{TokenCost} - \lambda_{\text{tool}} * \text{ToolCalls} - \lambda_{\text{latency}} * \text{Latency}$$

如果最终答案正确，`accuracy = 1`；错误则 `accuracy = 0` 或负分。

更简单的版本是：

Oracle path = 所有正确路径中 `cost` 最低的路径

例如：

样本	Direct	CoT	SQL	Oracle
A	正确，20 tokens	正确，180 tokens	正确，1 call	Direct
B	错误	正确，250 tokens	正确，1 call	SQL
C	错误	正确，220 tokens	SQL 报错	CoT
D	错误	错误	SQL repair 正确，2 calls	SQL-Repair

这样得到的 `supervision` 不是“哪个路径最强”，而是“哪个路径最划算”。

阶段三：训练 cost-aware router + executor

模型先输出路径，再执行对应推理：

```
<ROUTE> SQL </ROUTE>
<REASONING>
SELECT ...
</REASONING>
<ANSWER> ... </ANSWER>
```

或者：

```
<ROUTE> DIRECT </ROUTE>
<ANSWER> ... </ANSWER>
```

这里有两种实现方式。

方案 A：单模型端到端

同一个 Qwen3-1.7B 既负责 route，也负责生成推理轨迹。

优点：叙事最符合“内化”。

缺点：route 错了后续全错，训练更难。

方案 B：小 router + 多 executor

先训练一个轻量 router，选择 Direct / CoT / SQL / Fallback，然后调用不同 prompt 或模块。

优点：系统性能可能更强，分析清楚。

缺点：会被质疑不是完全内化。

我建议论文主线用 A，系统对比里保留 B。也就是：

- main method: single-model cost-aware path generation;
- ablation: external rule router / classifier router / oracle router.

6. 训练目标设计

可以分三层做，逐步增强。

6.1 Route-aware SFT

最基础：

Input: table + question

Output: <ROUTE> oracle_path </ROUTE> + trajectory + answer

这一步让模型先学会成本感知的路径选择。

6.2 Cost-aware DPO

构造 chosen / rejected:

chosen: 正确且低成本路径

rejected: 正确但高成本路径

这类 pair 很重要，因为普通 DPO 经常只区分对错，但 cost-aware DPO 要进一步区分：

都对的时候，选更便宜的。

例如：

chosen: <ROUTE> DIRECT </ROUTE> ... correct, 25 tokens

rejected: <ROUTE> COT </ROUTE> ... correct, 310 tokens

或者：

chosen: <ROUTE> SQL </ROUTE> ... correct, 1 tool call

rejected: <ROUTE> SQL_REPAIR </ROUTE> ... correct, 3 tool calls

也可以有准确率优先的 pair:

chosen: <ROUTE> SQL </ROUTE> correct, medium cost

rejected: <ROUTE> DIRECT </ROUTE> wrong, low cost

这样模型不会为了省成本乱答。

6.3 Budget-conditioned Training

这是更有新意的一步。

给模型一个预算条件：

```
<BUDGET> low </BUDGET>
```

或：

```
<BUDGET> max_tool_calls=0; max_tokens=100 </BUDGET>
```

让模型在不同预算下输出不同路径。

例如同一个问题：

低预算：

```
<ROUTE> DIRECT </ROUTE>
```

高预算：

```
<ROUTE> SQL_REPAIR </ROUTE>
```

这会让论文更有亮点：不是训练一个固定策略，而是训练一个**可控的成本-性能策略**。

7. 评价指标

这条线最重要的是评价指标要和 cost-aware 对齐，不要只报 accuracy。

7.1 基础性能

还是要报：

- Exact Match;
- F1;
- Rouge-L;
- TabFact accuracy;
- task-specific score。

你们现在已经有 WikiTableQA、TabFact、FeTaQA、HiTab 四个任务，可以沿用。

7.2 成本指标

必须新增：

指标	含义
Avg Output Tokens	平均生成 token
Avg Reasoning Tokens	reasoning 部分 token
Tool Calls / Query	每题工具调用次数
Repair Steps / Query	平均修复轮数
Latency / Query	平均耗时
SQL Execution Failure Rate	SQL 执行失败率

7.3 综合指标

可以定义：

$\text{Cost-normalized Accuracy} = \text{Accuracy} / \text{Average Cost}$

但这个指标有点粗。更推荐画 Pareto curve：

x 轴是 cost，y 轴是 accuracy。

比较：

- CoT-only;
- SQL-only;
- fixed SQL-first fallback;
- route-aware SFT;
- cost-aware DPO;
- oracle route。

如果你们的方法在同等成本下 accuracy 更高，或者同等 accuracy 下 cost 更低，这就是非常强的结果。

7.4 路由指标

指标	含义
Route Accuracy	是否选中 cost-aware oracle path
Oracle Gap	与 oracle routing 的性能差距
Overthinking Rate	简单题却走高成本路径的比例
Underthinking Rate	难题却走低成本路径导致错误的比例
Correct-but-expensive Rate	答对但路径不经济的比例

这几个指标会让论文分析非常清楚。

8. 关键实验设计

我建议主实验表不要只像现在一样报最终分数，而是改成这种结构：

Method	Avg Score	Avg Tokens	Tool Calls	Latency Cost@Acc
CoT-only		高	0	中
SQL-only		低/中	1+	中
SQL-first fallback		中高	高	高
Mixed SFT		中	中	中
Cost-aware SFT		更低	更低	更低
Cost-aware DPO		最优 trade-off		
Oracle route	上界	最低可行		

你们最想看到的结果不是 cost-aware 分数一定最高，而是：

在 accuracy 接近 mixed SFT 的情况下，token/tool/latency 显著降低。

例如：

Mixed SFT: 63.3 avg score, 420 tokens, 1.8 tool calls
Cost-aware SFT: 62.8 avg score, 210 tokens, 0.9 tool calls
Cost-aware DPO: 64.0 avg score, 230 tokens, 0.8 tool calls

这样的故事非常好讲。

9. Ablation 应该怎么做

我建议至少做这些消融。

9.1 没有成本监督

Mixed SFT

vs

Cost-aware route SFT

证明成本标签有用。

9.2 只用 route, 不用 DPO

Route-aware SFT

vs

Route-aware SFT + Cost-aware DPO

证明 preference optimization 有用。

9.3 不同 cost 权重

$\lambda = 0$

$\lambda = 0.1$

$\lambda = 0.3$

$\lambda = 0.5$

$\lambda = 1.0$

看 accuracy-cost trade-off。

9.4 不同预算模式

low budget

medium budget

high budget

证明模型能按预算改变路径选择。

9.5 Oracle routing

这是必须有的：

Oracle route

vs

Learned route

如果 gap 很小，说明模型学得好；如果 gap 大，说明还有提升空间。

10. 数据构造细节

你们可以用现有数据很快启动。

目前已有：

- v1: 单轮 CoT;
- v2: 多轮 SQL Agent;
- v5: SQL + CoT 回退;
- 四个任务的全量教师轨迹;
- Qwen3-1.7B 的 basic / cot / agent / mixed 权重。

在此基础上新增字段：

```
{
  "question_id": "...",
  "task": "HiTab",
  "paths": [
    {
      "type": "direct",
      "answer": "...",
      "correct": false,
      "token_cost": 34,
      "tool_calls": 0,
      "latency": 0.21
    },
    {
      "type": "cot",
      "answer": "...",
      "correct": false,
      "token_cost": 312,
      "tool_calls": 0,
      "latency": 1.32
    },
    {
      "type": "sql",
      "answer": "...",
      "correct": true,
      "token_cost": 156,
      "tool_calls": 1,
      "latency": 0.77
    }
  ]
}
```

```
],  
  "oracle_route": "sql"  
}
```

然后生成 SFT 输出：

```
<ROUTE> SQL </ROUTE>  
<SQL>  
SELECT ...  
</SQL>  
<EXECUTION_RESULT>  
...  
</EXECUTION_RESULT>  
<ANSWER>  
...  
</ANSWER>
```

DPO pair:

```
{  
  "prompt": "table + question",  
  "chosen": "<ROUTE> SQL ... correct medium cost",  
  "rejected": "<ROUTE> COT ... wrong high cost"  
}
```

或者：

```
{  
  "prompt": "table + question",  
  "chosen": "<ROUTE> DIRECT ... correct low cost",  
  "rejected": "<ROUTE> COT ... correct high cost"  
}
```

第二种尤其重要，因为它体现 cost-aware，而不是普通 accuracy-aware。

11. 这个方向和现有工作的区别

你们可以这样定位。

Table-R1 关注的是：

通过 reasoning trace distillation 和 RLVR 提升表格推理能力。

Table-r1 关注的是：

小模型 program-based table reasoning，并通过 mixed-paradigm GRPO 在需要时 fallback 到 text reasoning。

你们关注的是：

不同路径都有成本，表格推理应该学习“足够便宜且足够可靠”的路径选择策略。

也就是说，你们不是单纯问：

Which reasoning paradigm is better?

而是问：

Which reasoning paradigm is worth its cost for this query?

这就是差异。

此外，LLM routing 领域已经有一些 cost-aware routing 工作，例如 Cost-Spectrum Contrastive Routing 研究不同模型池中的成本敏感选择，Adaptive LLM Routing under Budget Constraints 把预算约束下的模型路由建模为多选背包问题。你们可以借这个 framing，但把它落到表格推理路径路由，不是“选哪个模型”。

12. 论文故事可以怎么写

可以这样组织。

Motivation

表格推理中的复杂路径不是免费的。CoT 消耗 token，SQL 需要工具调用，SQL Agent repair 增加延迟，hybrid fallback 增加系统复杂度。现有方法多追求最高 accuracy，却忽略了不同问题需要的推理成本差异。

Observation

不同任务和样本的最优路径不同：

- HiTab 数值聚合适合 SQL；

- FeTaQA 自由生成适合 CoT;
- 简单查找无需长 CoT 或 SQL;
- SQL 失败样本需要 fallback 或 repair。

你们报告里的 case study 已经支持这个观察：HiTab 上 SQL 计算准确，FeTaQA 上 CoT 更自然。

Method

提出 cost-aware path distillation:

1. 多路径教师轨迹生成;
2. 成本与正确性标注;
3. cost-aware oracle route 构造;
4. route-aware SFT;
5. cost-aware DPO;
6. budget-conditioned inference。

Result

展示:

- 同等 accuracy 下更低成本;
- 同等成本下更高 accuracy;
- route accuracy 高;
- overthinking rate 降低;
- 在小模型上尤其有效。

13. 可能的标题

几个可选标题:

1. Cost-aware Paradigm Distillation for Efficient Table Reasoning
2. Learning When to Think, Query, or Fallback: Cost-aware Routing for Small Table Reasoners
3. Cheap Enough Reasoning: Cost-aware Path Selection for Table Question Answering
4. From Hybrid Reasoning to Efficient Reasoning: Cost-aware Distillation for Table QA
5. Budgeted Table Reasoning with Small Language Models

我个人最喜欢第 2 个，比较直观：

Learning When to Think, Query, or Fallback

Think = CoT, Query = SQL, Fallback = 混合恢复。

14. 最小可行版本

如果时间有限，可以先做一个 MVP。

只保留三条路径：

Direct / CoT / SQL

成本只算两个：

output tokens + tool calls

oracle 定义为：

所有答对路径中 cost 最低者

训练只做：

route-aware SFT

评估只报：

accuracy、tokens、tool calls、route accuracy、oracle gap

这已经能形成一个完整故事。

之后再加 SQL-repair、fallback、DPO、budget-conditioned training。

15. 我建议你们接下来马上做的三个实验

第一，多路径结果表。

对同一批 100 或 500 样本，跑 Direct、CoT、SQL、Mixed，统计每条路径的 correctness 和 cost。

第二，oracle cost-aware routing 上界。

看每个样本选最低成本正确路径后，accuracy 和平均 cost 是多少。

第三，route-aware SFT 小实验。

训练模型输出 <ROUTE>，看它能否接近 oracle route。

如果这三个实验结果好，这个方向就很值得继续。

最终建议

我认为 cost-aware 这条线可以把你们的工作从“又一个 SQL+CoT 混合蒸馏”升级成：

面向部署的小模型表格推理：如何在正确率、token、工具调用和延迟之间学习最优路径选择。

这条线的好处是：

它利用了你们已有的 v1/v2/v5 轨迹资产；

它解释了为什么 mixed 有时好、有时不如 CoT；

它能自然引出 route-aware SFT / DPO；

它也让“1.7B 小模型可部署”这个卖点变得可量化。